

Tech & Teach

Digital Chip Design Program

RTL Design

Almas Alshubrumi

Atheer Al-harbi

Mjd AL-harbi

Maha Omar

Jood Alomair

15/7/2026

Sequential Datapath Design Part 3

Version 1.0

Tech & Teach

Task #9.1

```

`timescale 1ns/1ps
module Testbench;
parameter N = 4;
logic clk;
logic rstn;
logic en;
logic load;
logic [N-1:0] D;
logic [N-1:0] Q;
Counter #(N) DUT(
    .clk(clk),
    .rstn(rstn),
    .en(en),
    .load(load),
    .D(D),
    .Q(Q));
initial begin
    clk = 0;
    #5 clk = ~clk;
end
initial begin
    rstn = 0;
    en = 0;
    load = 0;
    D = 0;
    rstn = 1;
    en = 1;
    #50;
    load = 1;
    D = 4;
    #10;
    load = 0;
    #40;
    en = 0;
    #20;
    en = 1;
    #40;
    rstn = 0;
    #20;
    rstn = 1;
    #30;
    $finish;
end
endmodule

```

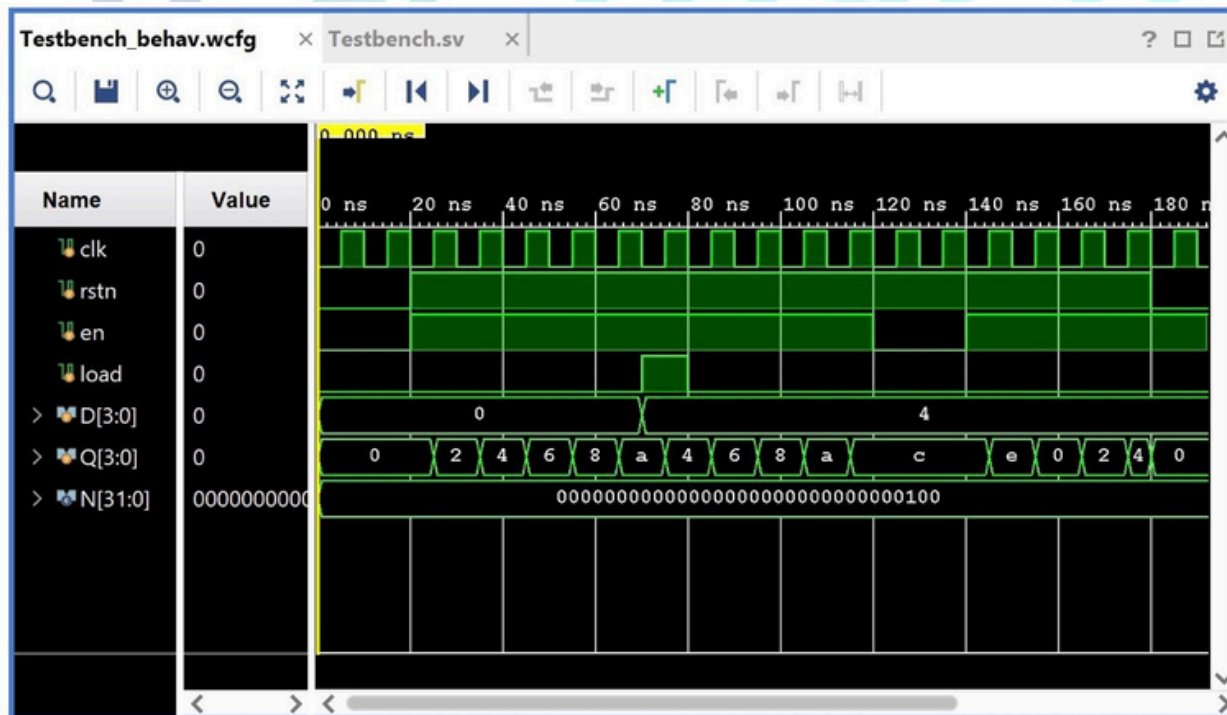
```

module Counter #(parameter N = 4)(
    input logic clk,
    input logic rstn,
    input logic en,
    input logic load,
    input logic [N-1:0] D,
    output logic [N-1:0] Q
);

always @(posedge clk or negedge rstn)
begin
    if(!rstn)
        Q <= 0;
    else if(load)
        Q <= D;
    else if(en)
        Q <= Q + 2;
end

endmodule

```



Task #9.2

```
`timescale 1ns/1ps
```

```
;module updown_counter_th
;logic [3:0] load_data
;logic clk
;logic rstn
;logic en
;logic load
;logic up_down
;logic [3:0] count
```

```
)updown_counter dut
,clk(clk).
,rstn(rstn).
,en(en).
,load(load).
,up_down(up_down).
,load_data(load_data).
,count(count).
;(  

;always #5 clk = ~clk
```

```
initial begin
```

```
;clk = 0
;rstn = 0
;en = 0
;load = 0
;up_down = 1
;load_data = 4'b0000
;#10
;rstn = 1
;en = 1; up_down = 1
;#10 ;#10 ;#10 ;#10 ;#10
```

```
;up_down = 0
;#10 ;#10
```

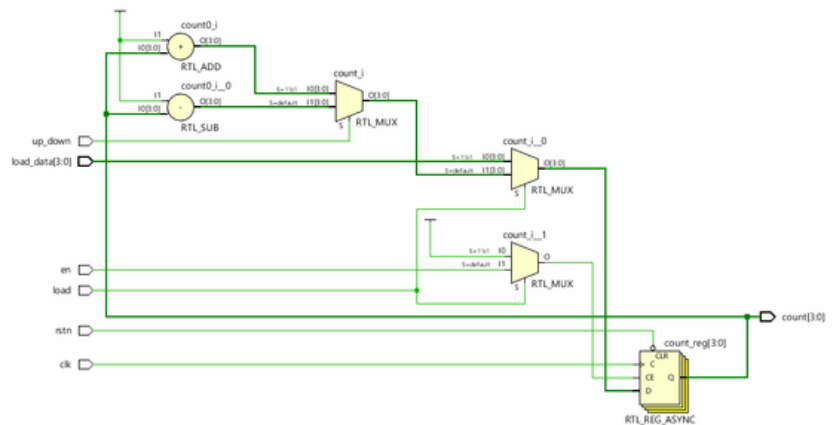
```
;en = 0
;load = 1; load_data = 4'b1010
;#10
;load = 0
```

```
;en = 1; up_down = 1
;#10
;rstn = 0
;#10
;rstn = 1
```

```
;#10
;finish$
end
endmodule
```

```
module updown_counter (
input logic clk,
input logic rstn,
input logic en,
input logic load,
input logic up_down,
input logic [3:0] load_data,
output logic [3:0] count
);
```

```
always @(posedge clk or negedge rstn) begin
if (!rstn)
count <= 4'b0000;
else if (load)
count <= load_data;
else if (en) begin
if (up_down)
count <= count + 1'b1;
else
count <= count - 1'b1;
end
end
endmodule
```



Task #9.3

```
module top #(parameter N = 10)(
    input logic clk,
    input logic rstn,
```

```
    output logic F
);
logic [$clog2(N)-1:0] w;
```

```
counter #($clog2(N)) count(
    .clk(clk),
    .rstn(rstn & ~(w == N)),
    .en(1),
    .l(0),
    .D(),
    .Q(w)
);
```

```
dff ff0(
    .clk(clk),
    .rstn(rstn),
    .en(w == N-1),
    .D(~F),
    .Q(F)
);
```

```
endmodule
```

```
`timescale 1ns / 1ps
module top_tb;
    parameter N = 10;
```

```
    logic clk;
    logic rstn;
```

```
    logic F;
```

```
    top #(N) dut(
        .clk(clk),
        .rstn(rstn),
        .F(F)
    );
```

```
    always begin
        #5 clk = ~clk;
    end
```

```
    initial begin
```

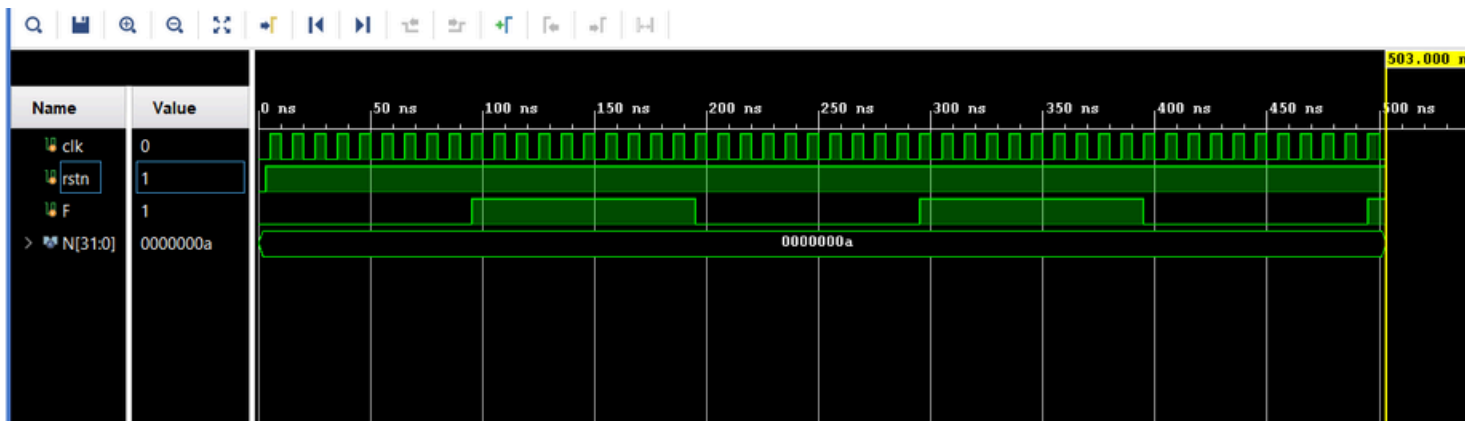
```
        clk = 0;
        rstn = 0;
```

```
        #3 rstn = 1;
```

```
        #500 $finish;
```

```
    end
```

```
endmodule
```



Work distribution

Section	Almas	Atheer	Mjd	Maha	Jood
Report	✓	✓	✓	✓	✓
Task9.1	✓	✓	✓	✓	✓
Task9.2	✓	✓	✓	✓	✓
Task9.3	✓	✓	✓	✓	✓

Tech & Teach